

6.3.3 乘法运算

1. 分析笔算乘法

$$A = -0.1101 \quad B = 0.1011$$

$$A \times B = -0.10001111 \quad \text{乘积的符号心算求得}$$

$$\begin{array}{r}
 0.1101 \\
 \times 0.1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 0.10001111
 \end{array}$$

✓ 符号位单独处理

✓ 乘数的某一位决定是否加被乘数

? 4个位积一起相加

✓ 乘积的位数扩大一倍

2. 笔算乘法改进

6.3

$$A \cdot B = A \cdot 0.1011$$

$$= 0.1A + 0.00A + 0.001A + 0.0001A$$

$$= 0.1A + 0.00A + 0.001(A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(A + 0.1A)]$$

右移一位

$$= 0.1\{A + 0.1[0 \cdot A + 0.1(A + 0.1A)]\}$$

$$= 2^{-1}\{A + 2^{-1}[0 \cdot A + 2^{-1}(A + 2^{-1}(A + 0))]\}$$

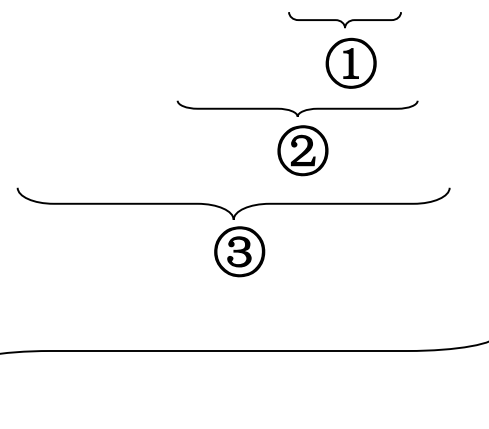
第一步 被乘数 $A + 0$

第二步 右移一位，得新的部分积

第三步 部分积 + 被乘数

⋮

第八步 右移一位，得结果



3. 改进后的笔算乘法过程（竖式） 6.3

部分积	乘数	说明
$\begin{array}{r} 0.0000 \\ + 0.1101 \\ \hline \end{array}$	$101\underline{1}$	初态，部分积 = 0 乘数为 1，加被乘数
$\begin{array}{r} 0.1101 \\ 0.0110 \\ + 0.1101 \\ \hline \end{array}$	$110\underline{1}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0011 \\ 0.1001 \\ + 0.0000 \\ \hline \end{array}$	$1\underline{110}$	→ 1，形成新的部分积 乘数为 0，加 0
$\begin{array}{r} 0.1001 \\ 0.0100 \\ + 0.1101 \\ \hline \end{array}$	$11\underline{11}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0001 \\ 0.1000 \\ \hline \end{array}$	111	→ 1，得结果

小结

- 乘法 运算可用 加和移位实现
 $n = 4$ ，加 4 次，移 4 次
- 由乘数的末位决定被乘数是否与原部分积相加，
然后 $\rightarrow 1$ 位形成新的部分积，同时 乘数 $\rightarrow 1$ 位
(末位移丢)，空出高位存放部分积的低位。
- 被乘数只与部分积的高位相加
硬件 3 个寄存器，具有移位功能
 1 个全加器

4. 原码乘法

(1) 原码一位乘运算规则

以小数为例

$$\text{设}[x]_{\text{原}} = x_0 \cdot x_1 x_2 \cdots x_n$$

$$[y]_{\text{原}} = y_0 \cdot y_1 y_2 \cdots y_n$$

$$\begin{aligned} [x \cdot y]_{\text{原}} &= (x_0 \oplus y_0) \cdot (0 \cdot x_1 x_2 \cdots x_n)(0 \cdot y_1 y_2 \cdots y_n) \\ &= (x_0 \oplus y_0) \cdot x^* y^* \end{aligned}$$

式中 $x^* = 0 \cdot x_1 x_2 \cdots x_n$ 为 x 的绝对值

$y^* = 0 \cdot y_1 y_2 \cdots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相乘 $x^* \cdot y^*$

(2) 原码一位乘递推公式

$$x^* \cdot y^* = x^*(0.y_1y_2 \dots y_n)$$

$$= x^*(y_12^{-1} + y_22^{-2} + \dots + y_n2^{-n})$$

$$= 2^{-1}(y_1x^* + 2^{-1}(y_2x^* + \dots 2^{-1}(y_nx^* + 0) \dots))$$

$$\underbrace{\dots \underbrace{\dots \underbrace{y_nx^* + 0}_{z_0}}_{z_1}}_{z_n}$$

$$z_0 = 0$$

$$z_1 = 2^{-1}(y_nx^* + z_0)$$

$$z_2 = 2^{-1}(y_{n-1}x^* + z_1)$$

$$\vdots$$

$$z_n = 2^{-1}(y_1x^* + z_{n-1})$$

例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$

6.3

解：

数值部分的运算

部分积	乘数	说明
0.0000 + 0.1110	110<u>1</u>	部分积 初态 $z_0 = 0$ + x^*
逻辑右移 0.1110 0.0111 + 0.0000	011<u>0</u>	$\rightarrow 1$, 得 z_1 + 0
逻辑右移 0.0111 0.0011 + 0.1110	0 101<u>1</u>	$\rightarrow 1$, 得 z_2 + x^*
逻辑右移 1.0001 0.1000 + 0.1110	10 110<u>1</u>	$\rightarrow 1$, 得 z_3 + x^*
逻辑右移 1.0110 0.1011	110 0110	$\rightarrow 1$, 得 z_4

例6.21 结果

6.3

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.10110110$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.10110110$$

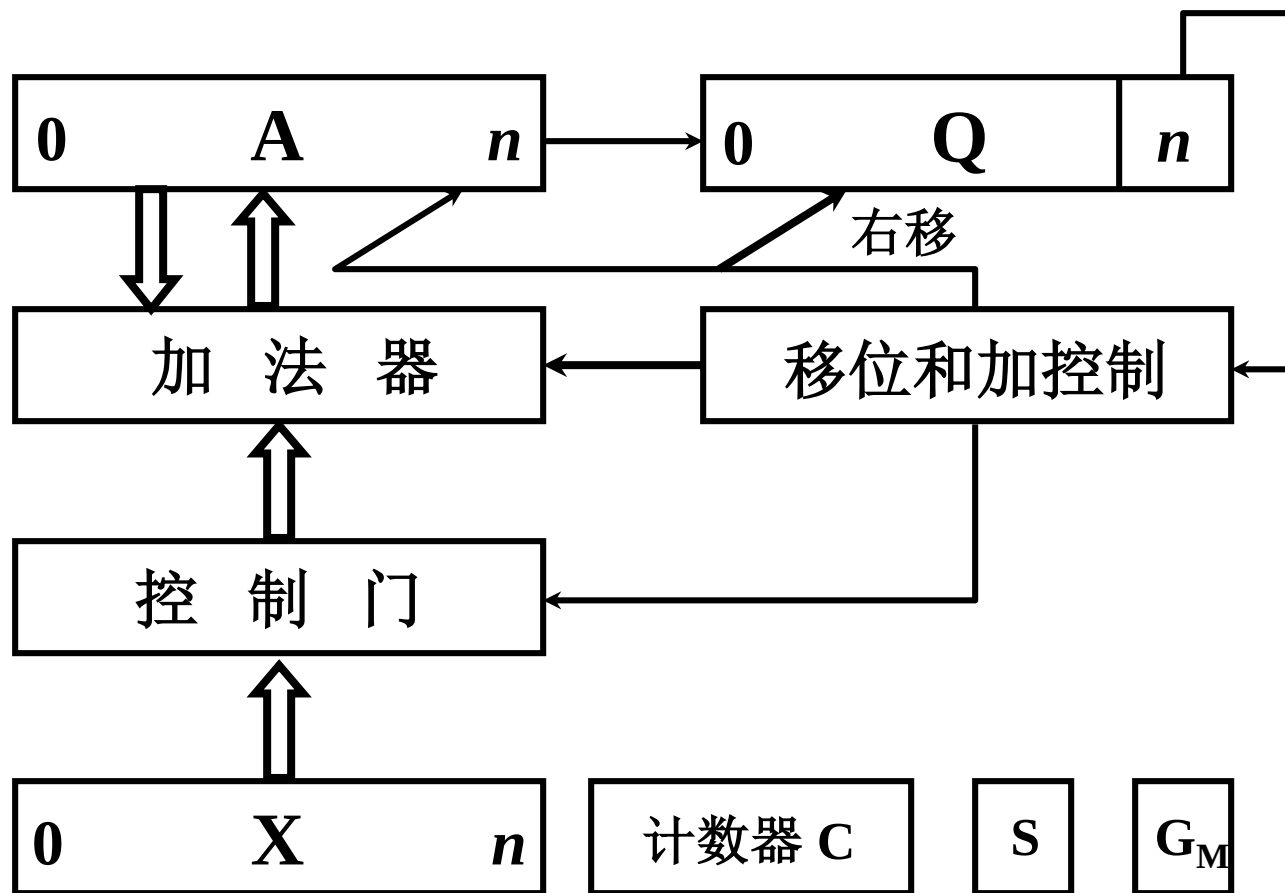
特点 绝对值运算

用移位的次数判断乘法是否结束

逻辑移位

(3) 原码一位乘的硬件配置

6.3



A、X、Q 均 $n+1$ 位

移位和加受末位乘数控制

(4) 原码两位乘

6.3

原码乘

符号位 和 数值位 部分 分开运算

两位乘

每次用 乘数的 2 位判断 原部分积
是否加 和 如何加 被乘数

乘数 $y_{n-1}y_n$	新的部分积
0 0	加 “0” → 2
0 1	加 1 倍的被乘数 → 2
1 0	加 2 倍的被乘数 → 2
1 1	加 3 倍的被乘数 → 2

$$\begin{array}{r} 3 ? \quad 4 \quad 100 \\ -1 \quad -01 \\ \hline 3 \quad 11 \end{array}$$

先 减 1 倍 的被乘数
再 加 4 倍 的被乘数

(5) 原码两位乘运算规则

6.3

乘数判断位 $y_{n-1}y_n$	标志位 C_j	操作内容
0 0	0	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
0 1	0	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 0	0	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 1	0	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “1”
0 0	1	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
0 1	1	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
1 0	1	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”
1 1	1	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”

共有操作	$+x^*$	$+2x^*$	$-x^*$	$\rightarrow 2$
实际操作	$+ [x^*]_{\text{补}}$	$+ [2x^*]_{\text{补}}$	$+ [-x^*]_{\text{补}}$	$\rightarrow 2$ 补码移

例6.22 已知 $x = 0.111111$ $y = -0.111001$ 求 $[x \cdot y]_{\text{原}}$

6.3

解: 数值部分的运算	乘数	C_j	说明
补码右移 000.000000 + 000.111111 000.111111 000.001111	00.111001 11 001110	0 0	初态 $z_0 = 0$ $+ x^*, C_j = 0$ $\rightarrow 2$ $+ 2x^*, C_j = 0$
补码右移 010.001101 000.100011 + 111.000001	11 0111 0011	0	$\rightarrow 2$ $- x^*, C_j = 1$
补码右移 111.100100 111.111001 + 000.111111 000.111000	0111 000111 00 000111	1	$\rightarrow 2$ $+ x^*, C_j = 0$

例6.22 结果

6.3

① 乘积的符号位 $x_0 \oplus y_0 = 0 \oplus 1 = 1$

② 数值部分的运算

$$x^* \cdot y^* = 0.111000000111$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.111000000111$$

特点 绝对值的补码运算

用移位的次数判断乘法是否结束

算术移位

(6) 原码两位乘和原码一位乘比较

	原码一位乘	原码两位乘
符号位	$x_0 \oplus y_0$	$x_0 \oplus y_0$
操作数	绝对值	绝对值的补码
移位	逻辑右移	算术右移
移位次数	n	$\frac{n}{2}$ (n 为偶数)
最多加法次数	n	$\frac{n}{2} + 1$ (n 为偶数)
思考	n 为奇数时, 原码两位乘 移? 次 最多加? 次	

5. 补码乘法

6.3

(1) 补码一位乘运算规则

以小数为例 设 被乘数 $[x]_{\text{补}} = x_0.x_1x_2 \cdots x_n$
乘数 $[y]_{\text{补}} = y_0.y_1y_2 \cdots y_n$

① 被乘数任意，乘数为正

同原码乘 但加和移位按补码规则运算
乘积的符号自然形成

② 被乘数任意，乘数为负

乘数 $[y]_{\text{补}}$ ，去掉符号位，操作同 ①

最后加 $[-x]_{\text{补}}$ ，校正

③ Booth 算法（被乘数、乘数符号任意） 6.3

设 $[x]_{\text{补}} = x_0 \cdot x_1 x_2 \cdots x_n$ $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \cdots y_n$

$[x \cdot y]_{\text{补}}$

$-[x]_{\text{补}} = +[-x]_{\text{补}}$

$$= [x]_{\text{补}} (0 \cdot y_1 \cdots y_n) - [x]_{\text{补}} \cdot y_0$$

$$= [x]_{\text{补}} (y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) - [x]_{\text{补}} \cdot y_0$$

$$2^{-1} = 2^0 - 2^{-1}$$

$$= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n})$$

$$2^{-2} = 2^{-1} - 2^{-2}$$

$$= [x]_{\text{补}} [-y_0 + (y_1 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \cdots + (y_n 2^{-(n-1)} - y_n 2^{-n})]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_n - y_{n-1}) 2^{-(n-1)} + (0 - y_n) 2^{-n}]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_{n+1} - y_n) 2^{-n}]$$

附加位 y_{n+1}

$$y_1 2^{-1} + \cdots + y_n 2^{-n}$$

④ Booth 算法递推公式

6.3

$$[z_0]_{\text{补}} = 0$$

$$[z_1]_{\text{补}} = 2^{-1}\{(y_{n+1}-y_n)[x]_{\text{补}} + [z_0]_{\text{补}}\} \quad y_{n+1} = 0$$

⋮

$$[z_n]_{\text{补}} = 2^{-1}\{(y_2-y_1)[x]_{\text{补}} + [z_{n-1}]_{\text{补}}\}$$

$$[x \cdot y]_{\text{补}} = [z_n]_{\text{补}} + (y_1-y_0)[x]_{\text{补}}$$

最后一步不移位

如何实现
 $y_{i+1}-y_i$?

y_i	y_{i+1}	$y_{i+1}-y_i$	操作
0	0	0	$\rightarrow 1$
0	1	1	$+ [x]_{\text{补}} \rightarrow 1$
1	0	-1	$+ [-x]_{\text{补}} \rightarrow 1$
1	1	0	$\rightarrow 1$

例6.23 已知 $x = +0.0011$ $y = -0.1011$ 求 $[x \cdot y]_{\text{补}}$ **6.3**

解:

0 0 . 0 0 0 0	1 . 0 1 0 <u>1</u>	<u>0</u>	
+ 1 1 . 1 1 0 1			$+[-x]_{\text{补}}$

$$[x]_{\text{补}} = 0.0011$$

$$[y]_{\text{补}} = 1.0101$$

$$[-x]_{\text{补}} = 1.1101$$

补码
右移

1 1 . 1 1 0 1			
1 1 . <u>1</u> 1 1 0	1	1 0 1 <u>0</u>	<u>1</u>
+ 0 0 . 0 0 1 1			$+ [x]_{\text{补}}$

补码
右移

0 0 . 0 0 0 1	1		
0 0 . <u>0</u> 0 0 0	1 1	1 0 <u>1</u>	<u>0</u>
+ 1 1 . 1 1 0 1			$+ [-x]_{\text{补}}$

补码
右移

1 1 . 1 1 0 1	1 1		
1 1 . <u>1</u> 1 1 0	1 1 1	1 <u>0</u>	<u>1</u>
+ 0 0 . 0 0 1 1			$+ [x]_{\text{补}}$

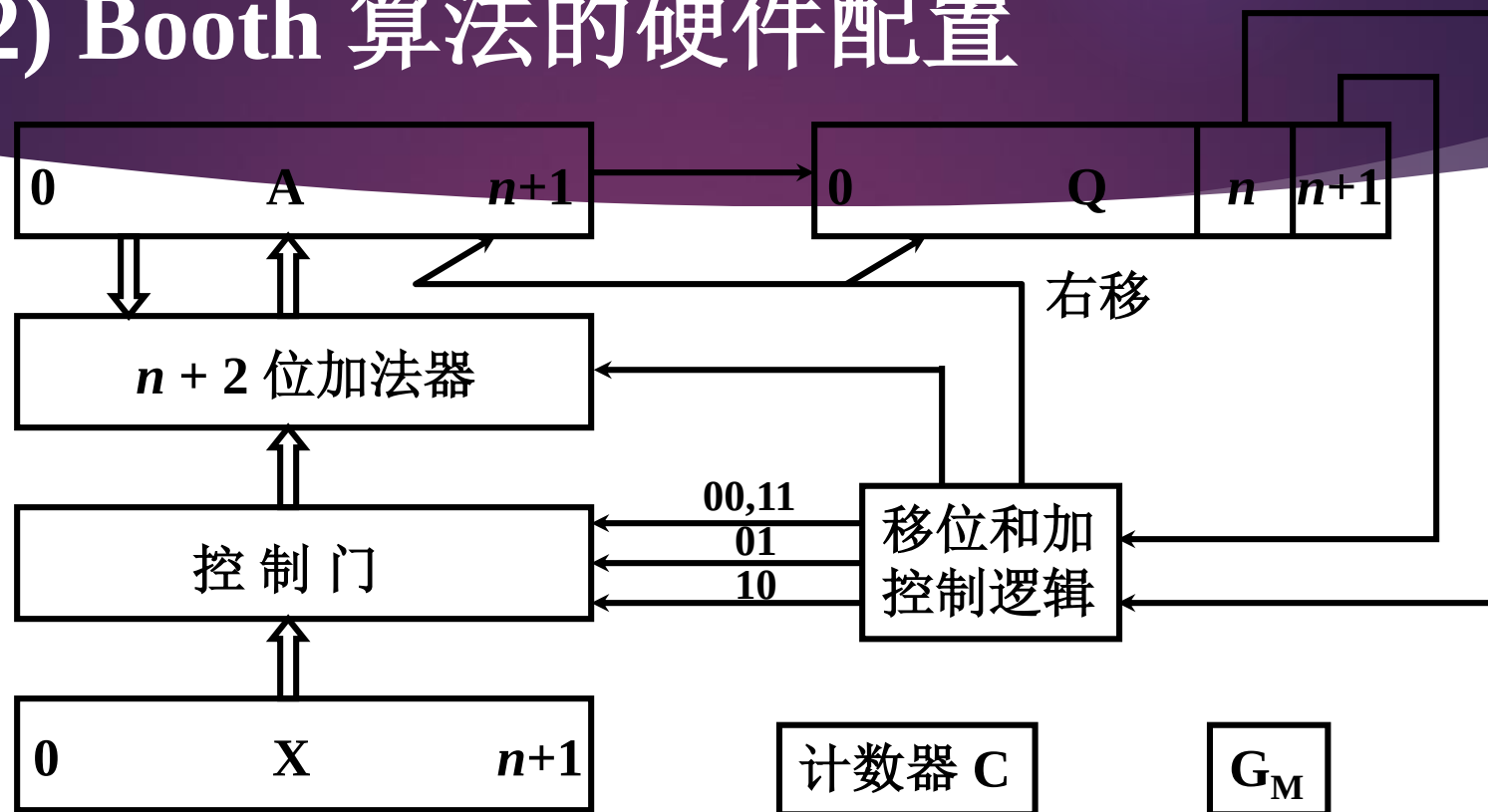
补码
右移

0 0 . 0 0 0 1	1 1 1		
0 0 . <u>0</u> 0 0 0	1 1 1 1	<u>1</u>	<u>0</u>
+ 1 1 . 1 1 0 1			$+ [-x]_{\text{补}}$

1 1 . 1 1 0 1	1 1 1 1		最后一步不移位
---------------	---------	--	---------

$$\therefore [x \cdot y]_{\text{补}} = 1.11011111$$

(2) Booth 算法的硬件配置



A、X、Q 均 $n+2$ 位

移位和加受末两位乘数控制

乘法小结

- 整数乘法与小数乘法完全相同
可用 逗号 代替小数点
- 原码乘 符号位 单独处理
补码乘 符号位 自然形成
- 原码乘去掉符号位运算 即为无符号数乘法
- 不同的乘法运算需有不同的硬件支持

四、除法运算

6.3

1. 分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

$$\begin{array}{r} 0.1101 \overline{) 0.10110} \\ \underline{0.01101} \\ 0.010010 \\ \underline{0.001101} \\ 0.00010100 \\ \underline{0.000001101} \\ 0.000000111 \end{array}$$

✓ 商符单独处理

? 心算上商

? 余数不动低位补“0”
减右移一位的除数

? 上商位置不固定

$$x \div y = -0.1101 \quad \text{商符心算求得}$$

$$\text{余数 } 0.000000111$$

2. 笔算除法和机器除法的比较

笔算除法

商符单独处理

心算上商

余数 不动 低位补“0”
减右移一位 的除数

2 倍字长加法器

上商位置 不固定

机器除法

符号位异或形成

$|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

余数 左移一位 低位补“0”
减 除数

1 倍字长加法器

在寄存器 最末位上商

3. 原码除法 以小数为例

$$[x]_{\text{原}} = x_0.x_1x_2 \dots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2 \dots y_n$$

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0) \cdot \frac{x^*}{y^*}$$

式中 $x^* = 0.x_1x_2 \dots x_n$ 为 x 的绝对值
 $y^* = 0.y_1y_2 \dots y_n$ 为 y 的绝对值

商的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相除 $\frac{x^*}{y^*}$

约定 小数定点除法 $x^* < y^*$ 整数定点除法 $x^* > y^*$

被除数不等于 0

除数不能为 0

(1) 恢复余数法

6.3

例6.24 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$

解: $[x]_{\text{原}} = 1.1011$ $[y]_{\text{原}} = 1.1101$ $[y^*]_{\text{补}} = 0.1101$ $[-y^*]_{\text{补}} = 1.0011$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

② 被除数 (余数)	商	说 明
0.1011	0.0000	
+ 1.0011		$+ [-y^*]_{\text{补}}$
1.1110	0	余数为负, 上商 0
+ 0.1101		恢复余数 $+ [y^*]_{\text{补}}$
0.1011	0	恢复后的余数
逻辑左移 1.0110	0	$\leftarrow 1$
+ 1.0011		$+ [-y^*]_{\text{补}}$
0.1001	01	余数为正, 上商 1
逻辑左移 1.0010	01	$\leftarrow 1$
+ 1.0011		$+ [-y^*]_{\text{补}}$

6.3

被除数（余数）	商	说 明
0.0101	011	余数为正，上商 1
逻辑左移 0.1010	011	← 1
+ 1.0011		+ $[-y^*]_{\text{补}}$
1.1101	0110	余数为负，上商 0
+ 0.1101		恢复余数 + $[y^*]_{\text{补}}$
0.1010	0110	恢复后的余数
逻辑左移 1.0100	0110	← 1
+ 1.0011		+ $[-y^*]_{\text{补}}$
0.0111	01101	余数为正，上商 1

$$\therefore \left[\frac{x}{y} \right]_{\text{原}} = 0.1101$$

余数为正 上商 1

余数为负 上商 0，恢复余数

上商 5 次

第一次上商判溢出

移 4 次

(2) 不恢复余数法（加减交替法）

- 恢复余数法运算规则

余数 $R_i > 0$ 上商 “1”， $2R_i - y^*$

余数 $R_i < 0$ 上商 “0”， $R_i + y^*$ 恢复余数

- 不恢复余数法运算规则 加减交替

上商 “1” $2R_i - y^*$

上商 “0” $2R_i + y^*$

$$2(R_i + y^*) - y^* = 2R_i + y^*$$

例6.25 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$ 6.3

解:

0 . 1 0 1 1	0 . 0 0 0 0	
+ 1 . 0 0 1 1		$+[-y^*]_{\text{补}}$
1 . 1 1 1 0	0	余数为负, 上商 0
1 . 1 1 0 0	0	$\leftarrow 1$
+ 0 . 1 1 0 1		$+ [y^*]_{\text{补}}$
0 . 1 0 0 1	0 1	余数为正, 上商 1
1 . 0 0 1 0	0 1	$\leftarrow 1$
+ 1 . 0 0 1 1		$+ [-y^*]_{\text{补}}$
0 . 0 1 0 1	0 1 1	余数为正, 上商 1
0 . 1 0 1 0	0 1 1	$\leftarrow 1$
+ 1 . 0 0 1 1		$+ [-y^*]_{\text{补}}$
1 . 1 1 0 1	0 1 1 0	余数为负, 上商 0
1 . 1 0 1 0	0 1 1 0	$\leftarrow 1$
+ 0 . 1 1 0 1		$+ [y^*]_{\text{补}}$
0 . 0 1 1 1	0 1 1 0 1	余数为正, 上商 1

逻辑左移
逻辑左移
逻辑左移
逻辑左移
逻辑左移

$$[x]_{\text{原}} = 1.1011$$

$$[y]_{\text{原}} = 1.1101$$

$$[x^*]_{\text{补}} = 0.1011$$

$$[y^*]_{\text{补}} = 0.1101$$

$$[-y^*]_{\text{补}} = 1.0011$$

例6.25 结果

6.3

$$\textcircled{1} x_0 \oplus y_0 = 1 \oplus 1 = 0$$

$$\textcircled{2} \frac{x^*}{y^*} = 0.1101$$

$$\therefore \left[\frac{x}{y} \right]_{\text{原}} = 0.1101$$

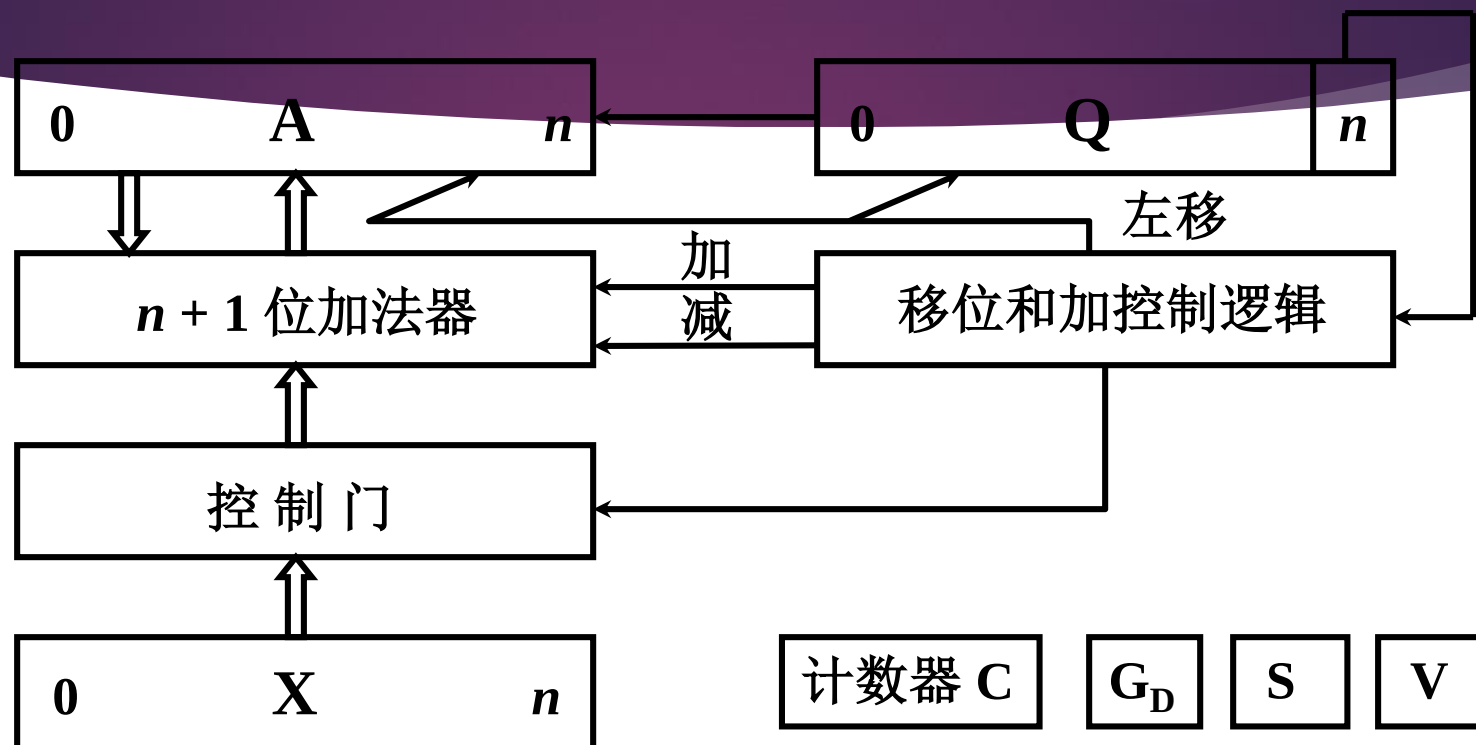
特点 上商 $n+1$ 次

第一次上商判溢出

移 n 次，加 $n+1$ 次

用移位的次数判断除法是否结束

(3) 原码加减交替除法硬件配置



A、X、Q 均 $n+1$ 位
用 Q_n 控制加减交替

4. 补码除法

6.3

(1) 商值的确定

① 比较被除数和除数绝对值的大小

x 与 y 同号

$$x = 0.1011 \quad [x]_{\text{补}} = 0.1011$$

$$y = 0.0011 \quad [y]_{\text{补}} = \boxed{0}.0011$$

$$[x]_{\text{补}} = 0.1011$$

$$+ [-y]_{\text{补}} = 1.1101$$

$$\hline [R_i]_{\text{补}} = \boxed{0}.1000$$

$$x^* > y^*$$

$$[R_i]_{\text{补}} \text{ 与 } [y]_{\text{补}} \text{ 同号}$$

“够减”

$$x = -0.0011 \quad [x]_{\text{补}} = 1.1101$$

$$y = -0.1011 \quad [y]_{\text{补}} = \boxed{1}.0101$$

$$[x]_{\text{补}} = 1.1101$$

$$+ [-y]_{\text{补}} = 0.1011$$

$$\hline [R_i]_{\text{补}} = \boxed{0}.1000$$

$$x^* < y^*$$

$$[R_i]_{\text{补}} \text{ 与 } [y]_{\text{补}} \text{ 异号}$$

“不够减”

6.3

x 与 y 异号

$$x = 0.1011 \quad [x]_{\text{补}} = 0.1011$$

$$y = -0.0011 \quad [y]_{\text{补}} = \boxed{1.1101}$$

$$\begin{array}{r} [x]_{\text{补}} = 0.1011 \\ + [y]_{\text{补}} = 1.1101 \\ \hline [R_i]_{\text{补}} = \boxed{0.1000} \end{array}$$

$$x^* > y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号

“够减”

$$x = -0.0011 \quad [x]_{\text{补}} = 1.1101$$

$$y = 0.1011 \quad [y]_{\text{补}} = \boxed{0.1011}$$

$$\begin{array}{r} [x]_{\text{补}} = 1.1101 \\ + [y]_{\text{补}} = 0.1011 \\ \hline [R_i]_{\text{补}} = \boxed{0.1000} \end{array}$$

$$x^* < y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号

“不够减”

小结

$[x]_{\text{补}}$ 和 $[y]_{\text{补}}$	求 $[R_i]_{\text{补}}$	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$
同号	$[x]_{\text{补}} - [y]_{\text{补}}$	同号, “够减”
异号	$[x]_{\text{补}} + [y]_{\text{补}}$	异号, “够减”

② 商值的确定 末位恒置“1”法

6.3

	$\times.\times\times\times\times 1$		
$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号 正商	$0.\underbrace{\times\times\times\times}_{\text{原码}}1$	按原码上商	“够减”上“1” “不够减”上“0”
$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号 负商	$1.\underbrace{\times\times\times\times}_{\text{反码}}1$	按反码上商	“够减”上“0” “不够减”上“1”

小 结

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$	商	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商 值
同 号	正	够减 (同号)	1
		不够减 (异号)	0
异 号	负	够减 (异号)	0
		不够减 (同号)	1

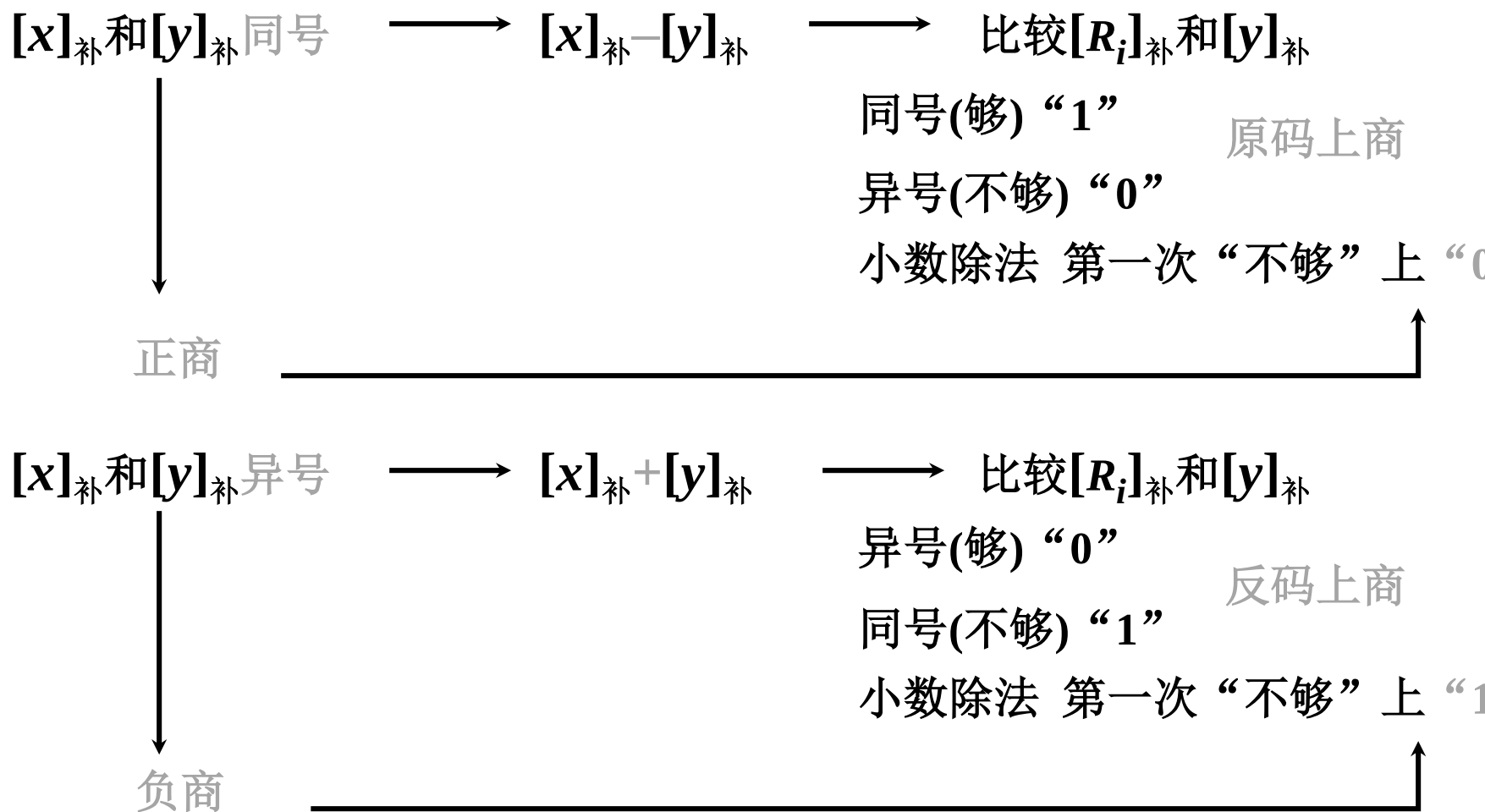
简 化 为

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同 号	1
异 号	0

(2) 商符的形成

6.3

除法过程中自然形成



(3) 新余数的形成

加减交替

$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$	商	新余数
同 号	1	$2[R_i]_{\text{补}} + [-y]_{\text{补}}$
异 号	0	$2[R_i]_{\text{补}} + [y]_{\text{补}}$

例6.26 设 $x = -0.1011$ $y = 0.1101$
求 $[\frac{x}{y}]_{\text{补}}$ 并还原成真值

解: $[x]_{\text{补}} = 1.0101$ $[y]_{\text{补}} = 0.1101$ $[-y]_{\text{补}} = 1.0011$

	1 . 0 1 0 1	0 . 0 0 0 0	
逻辑左移	+ 0 . 1 1 0 1		异号做加法
	0 . 0 0 1 0	1	同号上“1”
	0 . 0 1 0 0	1	←1
	+ 1 . 0 0 1 1		+ $[-y]_{\text{补}}$
逻辑左移	1 . 0 1 1 1	1 0	异号上“0”
	0 . 1 1 1 0	1 0	←1
	+ 0 . 1 1 0 1		+ $[y]_{\text{补}}$
逻辑左移	1 . 1 0 1 1	1 0 0	异号上“0”
	1 . 0 1 1 0	1 0 0	←1
	+ 0 . 1 1 0 1		+ $[y]_{\text{补}}$
逻辑左移	0 . 0 0 1 1	1 0 0 1	同号上“1”
	0 . 0 1 1 0	1 0 0 1 1	←1 末位恒置“1”

$$\therefore [\frac{x}{y}]_{\text{补}} = 1.0011$$

$$\text{则 } \frac{x}{y} = -0.1101$$

(4) 小结

- 补码除法共上商 $n+1$ 次（末位恒置 1）
第一次为商符
- 第一次商可判溢出
- 加 n 次 移 n 次
- 用移位的次数判断除法是否结束
- 精度误差最大为 2^{-n}

(5) 补码除和原码除（加减交替法）比较 6.3

	原码除	补码除
商符	$x_0 \oplus y_0$	自然形成
操作数	绝对值补码	补码
上商原则	余数的正负	比较余数和除数的符号
上商次数	$n + 1$	$n + 1$
加法次数	$n + 1$	n
移位	逻辑左移	逻辑左移
移位次数	n	n
第一步操作	$[x^*]_{\text{补}} - [y^*]_{\text{补}}$	同号 $[x]_{\text{补}} - [y]_{\text{补}}$ 异号 $[x]_{\text{补}} + [y]_{\text{补}}$

6.4 浮点四则运算

6.4.1 浮点加减运算

1. 对阶 $x = S_x \cdot 2^{j_x}$ $y = S_y \cdot 2^{j_y}$

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y & \text{已对齐} \\ > 0 & j_x > j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1, j_x + 1 \\ y \text{ 向 } x \text{ 看齐} & \checkmark S_y \rightarrow 1, j_x - 1 \end{cases} \\ < 0 & j_x < j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & \checkmark S_x \rightarrow 1, j_y - 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1, j_y + 1 \end{cases} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐

例如 $x = 0.1101 \times 2^{01}$ $y = (-0.1010) \times 2^{11}$ **6.4**

求 $x+y$

解: $[x]_{\text{补}} = 00, 01; 00.1101$ $[y]_{\text{补}} = 00, 11; 11.0110$

1. 对阶

① 求阶差 $[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 00, 01$

$$\begin{array}{r} + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

阶差为负 (-2) $\therefore S_x \rightarrow 2 \quad j_x + 2$

② 对阶 $[x]_{\text{补}'} = 00, 11; 00.0011$

2. 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}'} = 00.0011 \quad \text{对阶后的}[S_x]_{\text{补}'} \\ + \quad [S_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \\ \therefore [x+y]_{\text{补}} = 00, 11; 11. 1001 \end{array}$$

3. 规格化

6.4

(1) 规格化数的定义

$$r = 2 \quad \frac{1}{2} \leq |S| < 1$$

(2) 规格化数的判断

$S > 0$	规格化形式	$S < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	$0.\boxed{1} \times \times \dots \times$	原码	$1.\boxed{1} \times \times \dots \times$
补码	$\boxed{0.1} \times \times \dots \times$	补码	$\boxed{1.0} \times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同

特例

$$S = -\frac{1}{2} = -0.100 \dots 0$$

$$[S]_{\text{原}} = 1.100 \dots 0$$

$$[S]_{\text{补}} = \boxed{1.1}00 \dots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$S = -1$$

$$[S]_{\text{补}} = \boxed{1.0}00 \dots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数

(3) 左规

尾数左移一位，阶码减 1，直到数符和第一数位不同为止

上例 $[x+y]_{\text{补}} = 00, 11; 11. 1001$

左规后 $[x+y]_{\text{补}} = 00, 10; 11. 0010$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

(4) 右规

当尾数溢出（>1）时，需右规

即尾数出现 $01. \times \times \dots \times$ 或 $10. \times \times \dots \times$ 时

尾数右移一位，阶码加 1

例6.27 $x = 0.1101 \times 2^{10}$ $y = 0.1011 \times 2^{01}$ **6.4**

求 $x+y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $[x]_{\text{补}} = 00, 010; 00. 110100$
 $[y]_{\text{补}} = 00, 001; 00. 101100$

① 对阶

$$[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = \begin{array}{r} 00, 010 \\ + 11, 111 \\ \hline 100, 001 \end{array}$$

阶差为 +1 $\therefore S_y \rightarrow 1, j_y+1$

$\therefore [y]_{\text{补}}' = 00, 010; 00. 010110$

② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}} = 00. 110100 \\ + [S_y]_{\text{补}}' = 00. 010110 \\ \hline 01. 001010 \end{array} \quad \begin{array}{l} \text{对阶后的 } [S_y]_{\text{补}}' \\ \text{尾数溢出需右规} \end{array}$$

③ 右规

$$[x+y]_{\text{补}} = 00, 010; 01. 001010$$

右规后

$$[x+y]_{\text{补}} = 00, 011; 00. 100101$$

$$\therefore x+y = 0. 100101 \times 2^{11}$$

4. 舍入

在对阶和右规过程中，可能出现尾数末位丢失引起误差，需考虑舍入

(1) 0 舍 1 入法

(2) 恒置 “1” 法

6.4

例 6.28 $x = (-\frac{5}{8}) \times 2^{-5}$ $y = (-\frac{7}{8}) \times 2^{-4}$

求 $x-y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $x = (-0.101000) \times 2^{-101}$ $y = (0.111000) \times 2^{-100}$

$[x]_{\text{补}} = 11, 011; 11. 011000$ $[y]_{\text{补}} = 11, 100; 00. 111000$

① 对阶

$$\begin{array}{rcl} [\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} & = & 11, 011 \\ & + & 00, 100 \\ \hline & & 11, 111 \end{array}$$

阶差为 -1 $\therefore S_x \longrightarrow 1, j_x + 1$

$\therefore [x]_{\text{补}} = 11, 100; 11. 101100$

② 尾数求和

$$\begin{array}{r}
 [S_x]_{\text{补}'} = 11.101100 \\
 + [-S_y]_{\text{补}} = 11.001000 \\
 \hline
 110.110100
 \end{array}$$

③ 右规

$$[x - y]_{\text{补}} = 11, 100; 10.110100$$

右规后

$$[x - y]_{\text{补}} = 11, 101; 11.011010$$

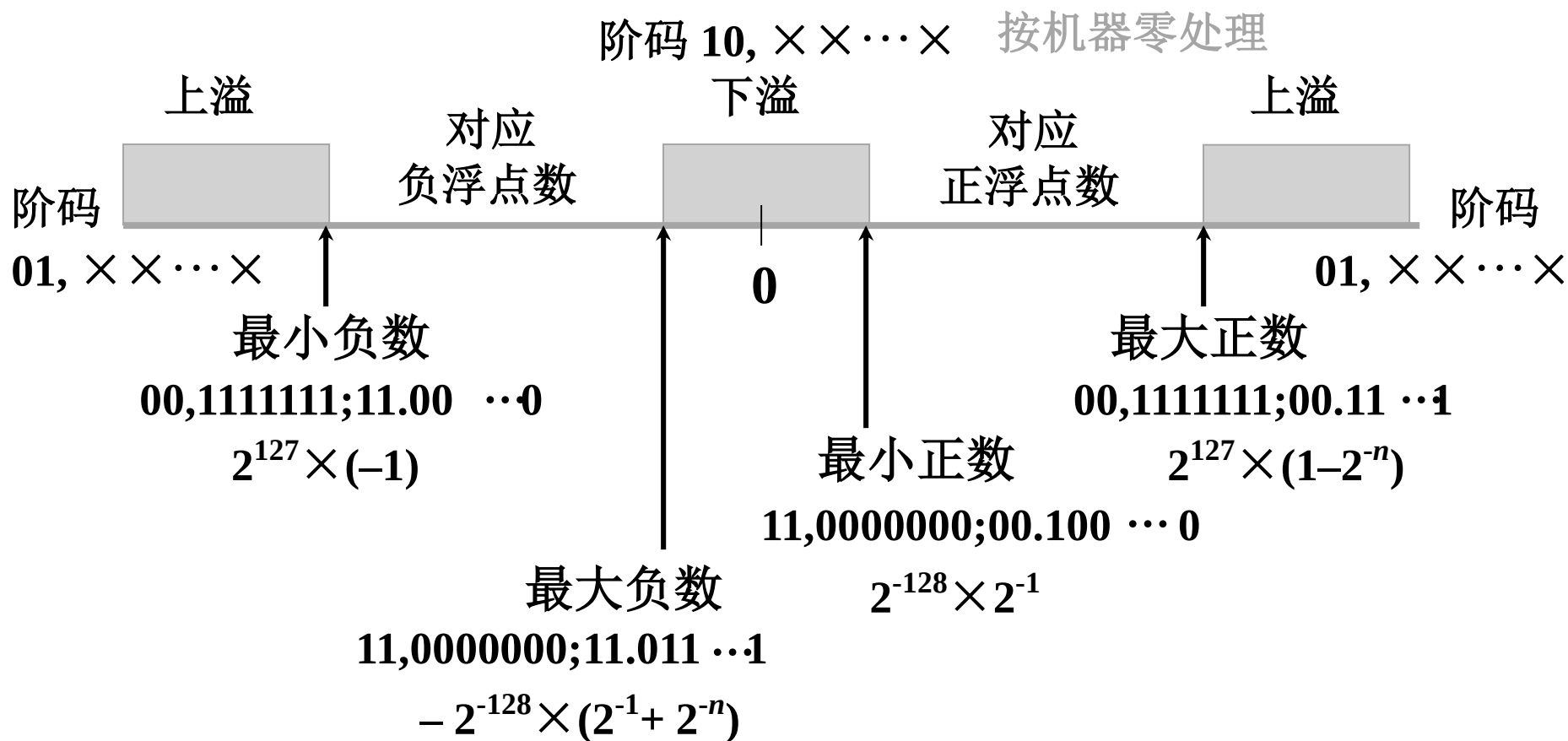
$$\therefore x - y = (-0.100110) \times 2^{-11}$$

$$= \left(-\frac{19}{32}\right) \times 2^{-3}$$

5. 溢出判断

6.4

设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码的数值部分取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为



二、浮点乘除运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 乘法

$$x \cdot y = (S_x \cdot S_y) \times 2^{j_x + j_y}$$

2. 除法

$$\frac{x}{y} = \frac{S_x}{S_y} \times 2^{j_x - j_y}$$

3. 步骤

(1) 阶码采用 补码定点加（乘法）减（除法）运算

(2) 尾数乘除同 定点 运算

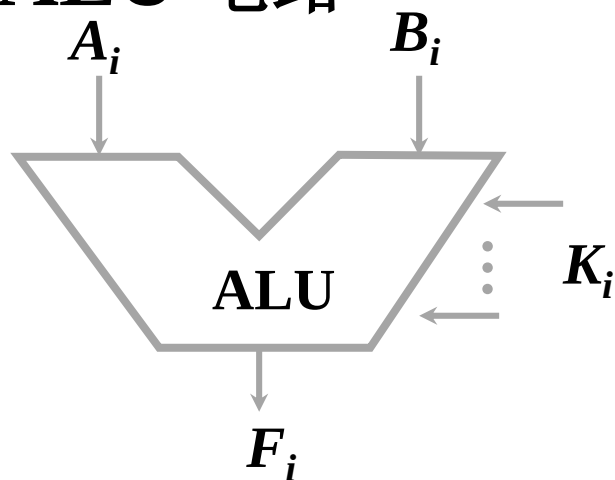
(3) 规格化

4. 浮点运算部件

阶码运算部件，尾数运算部件

6.5 算术逻辑单元

6.5.1 ALU 电路



组合逻辑电路

K_i 不同取值

F_i 不同

四位 ALU 74181

$M = 0$ 算术运算

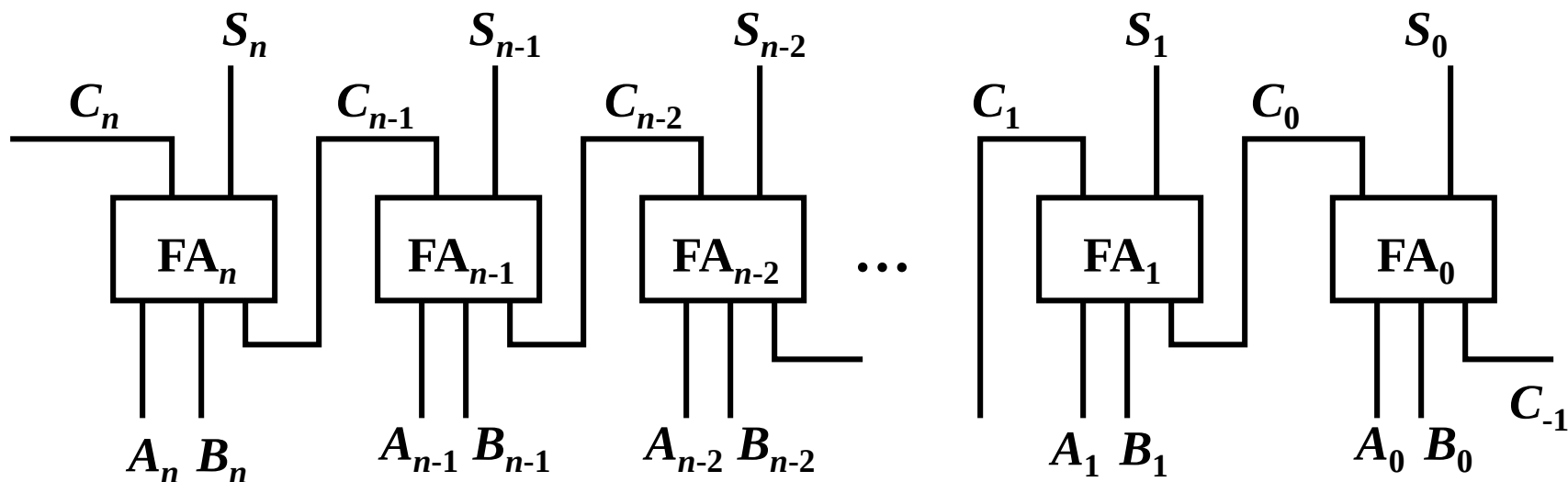
$M = 1$ 逻辑运算

$S_3 \sim S_0$ 不同取值, 可做不同运算

二、快速进位链

6.5

1. 并行加法器



$$S_i = \overline{A_i} \overline{B_i} C_{i-1} + \overline{A_i} B_i \overline{C_{i-1}} + A_i \overline{B_i} \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$C_i = \overline{A_i} B_i C_{i-1} + A_i \overline{B_i} C_{i-1} + A_i B_i \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$= A_i B_i + (A_i + B_i) C_{i-1}$$

$$d_i = A_i B_i \quad \text{本地进位}$$

$$t_i = A_i + B_i \quad \text{传送条件}$$

则 $C_i = d_i + t_i C_{i-1}$

2. 串行进位链

6.5

进位链

传送进位的电路

串行进位链

进位串行传送

以 4 位全加器为例，每一位的进位表达式为

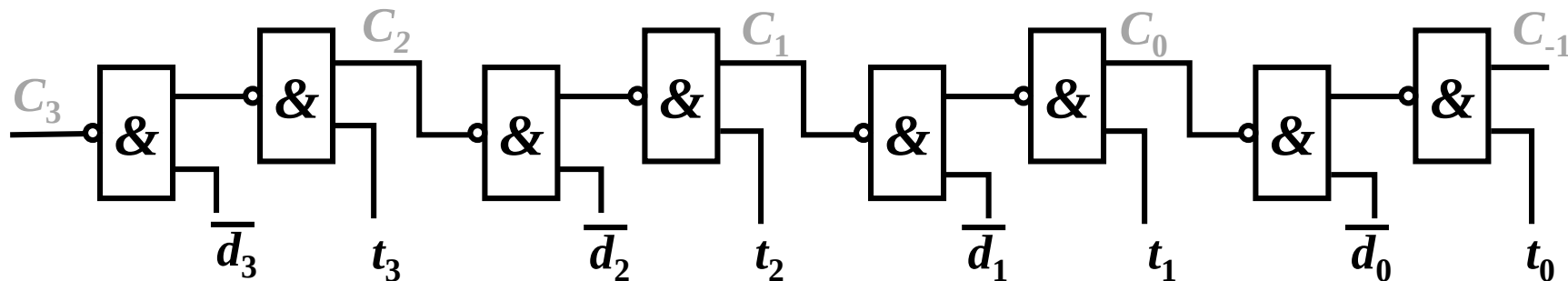
$$C_0 = d_0 + t_0 C_{-1} = \overline{\overline{d_0} \cdot \overline{t_0 C_{-1}}}$$

$$C_1 = d_1 + t_1 C_0$$

$$C_2 = d_2 + t_2 C_1$$

$$C_3 = d_3 + t_3 C_2$$

设与非门的级延迟时间为 t_y



4 位全加器产生进位的全部时间为 $8t_y$

n 位全加器产生进位的全部时间为 $2nt_y$

3. 并行进位链（先行进位，跳跃进位）

6.5

n 位加法器的进位同时产生 以 4 位加法器为例

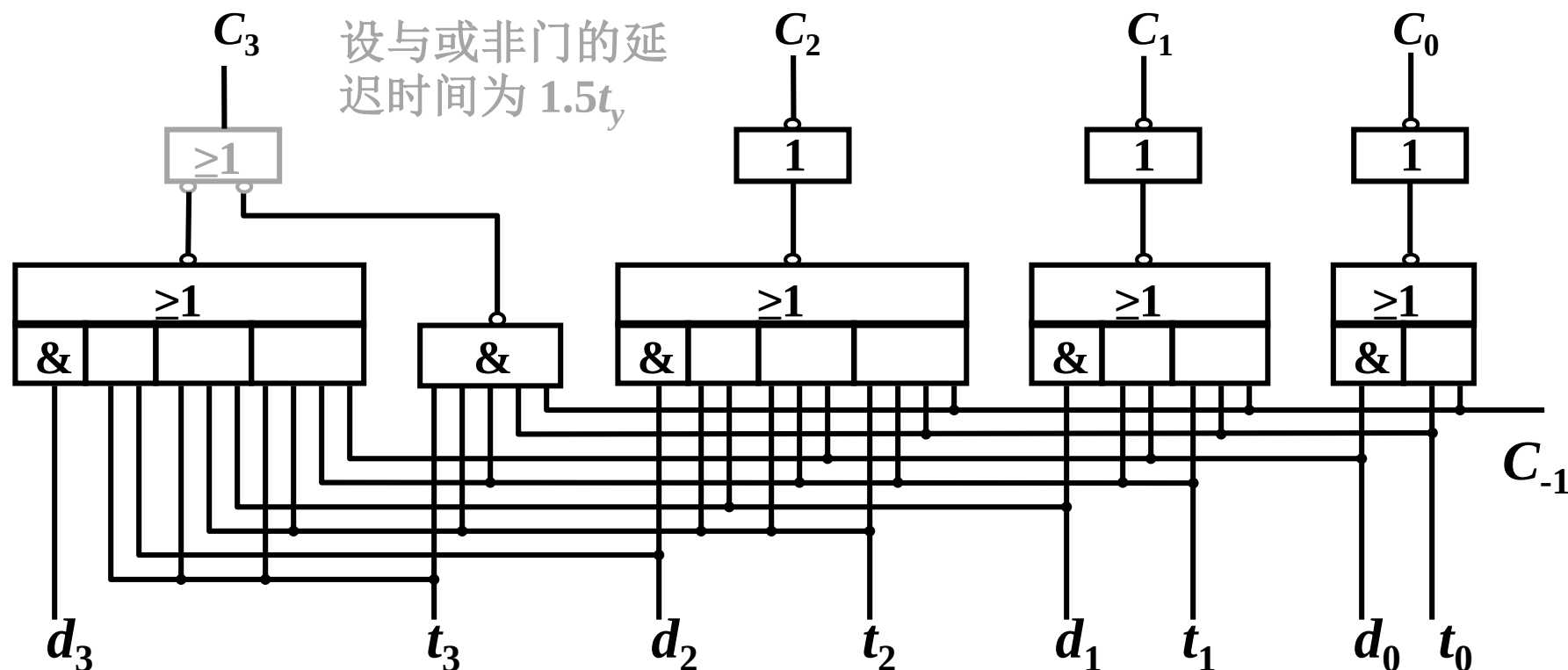
$$C_0 = d_0 + t_0 C_{-1}$$

$$C_1 = d_1 + t_1 C_0 = d_1 + t_1 d_0 + t_1 t_0 C_{-1}$$

$$C_2 = d_2 + t_2 C_1 = d_2 + t_2 d_1 + t_2 t_1 d_0 + t_2 t_1 t_0 C_{-1}$$

$$C_3 = d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_{-1}$$

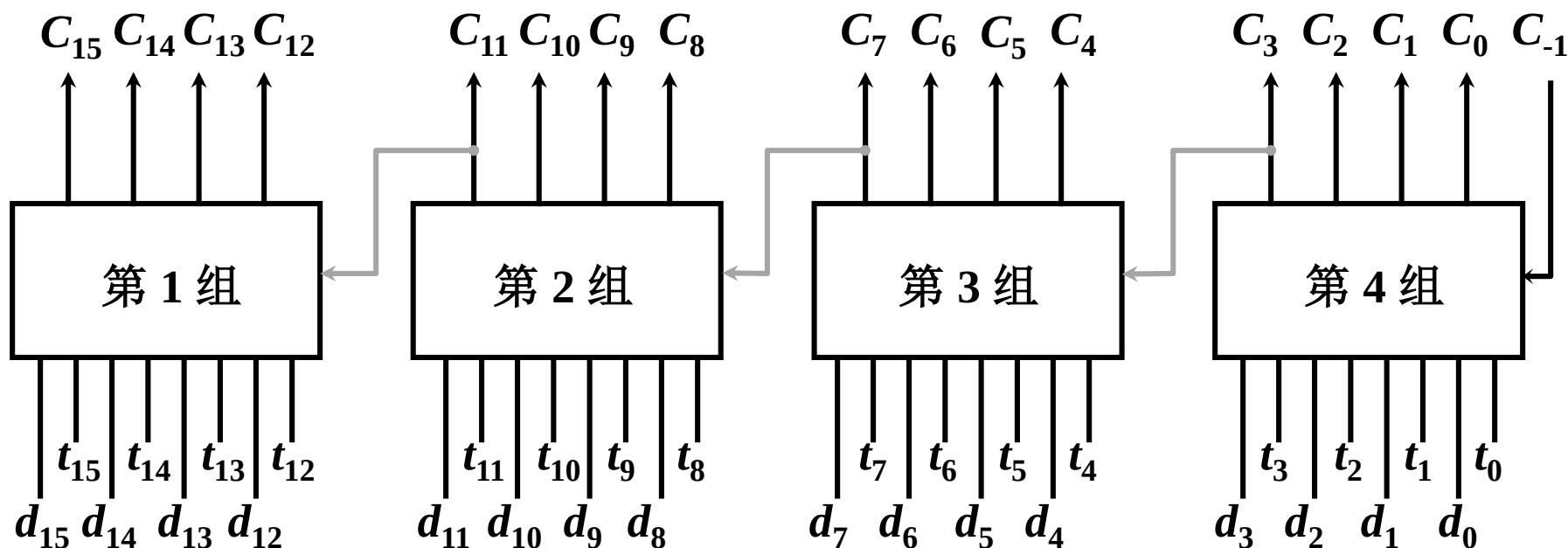
当 $d_i t_i$ 形成后，只需 $2.5t_y$
产生全部进位



(1) 单重分组跳跃进位链

6.5

n 位全加器分若干小组，小组中的进位同时产生，
小组与小组之间采用串行进位 以 $n = 16$ 为例



当 $d_i t_i$ 形成后

经 $2.5 t_y$

$5 t_y$

$7.5 t_y$

$10 t_y$

产生 $C_3 \sim C_0$

产生 $C_7 \sim C_4$

产生 $C_{11} \sim C_8$

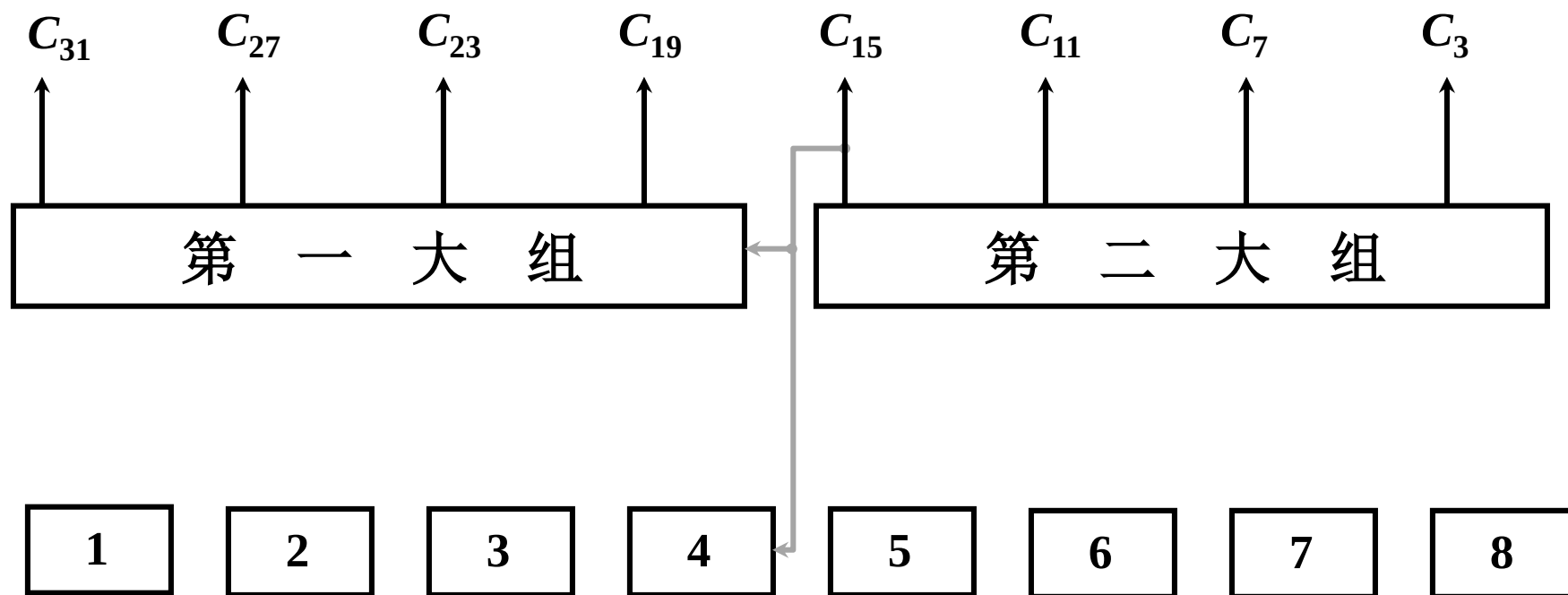
产生 $C_{15} \sim C_{12}$

(2) 双重分组跳跃进位链

6.5

n 位全加器分若干大组，大组中又包含若干小组。每个大组中小组的最高位进位同时产生。大组与大组之间采用串行进位。

以 $n = 32$ 为例



(3) 双重分组跳跃进位链 大组进位分析

6.5

以第 8 小组为例

$$\begin{aligned} C_3 &= d_3 + t_3 C_2 = \underbrace{d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0}_{D_8} + \underbrace{t_3 t_2 t_1 t_0 C_{-1}}_{T_8 C_{-1}} \\ &= D_8 + T_8 C_{-1} \end{aligned}$$

D_8 小组的本地进位 与外来进位无关

T_8 小组的传送条件 与外来进位无关 传递外来进位

同理 第 7 小组 $C_7 = D_7 + T_7 C_3$

第 6 小组 $C_{11} = D_6 + T_6 C_7$

第 5 小组 $C_{15} = D_5 + T_5 C_{11}$

进一步展开得

$$C_3 = D_8 + T_8 C_{-1}$$

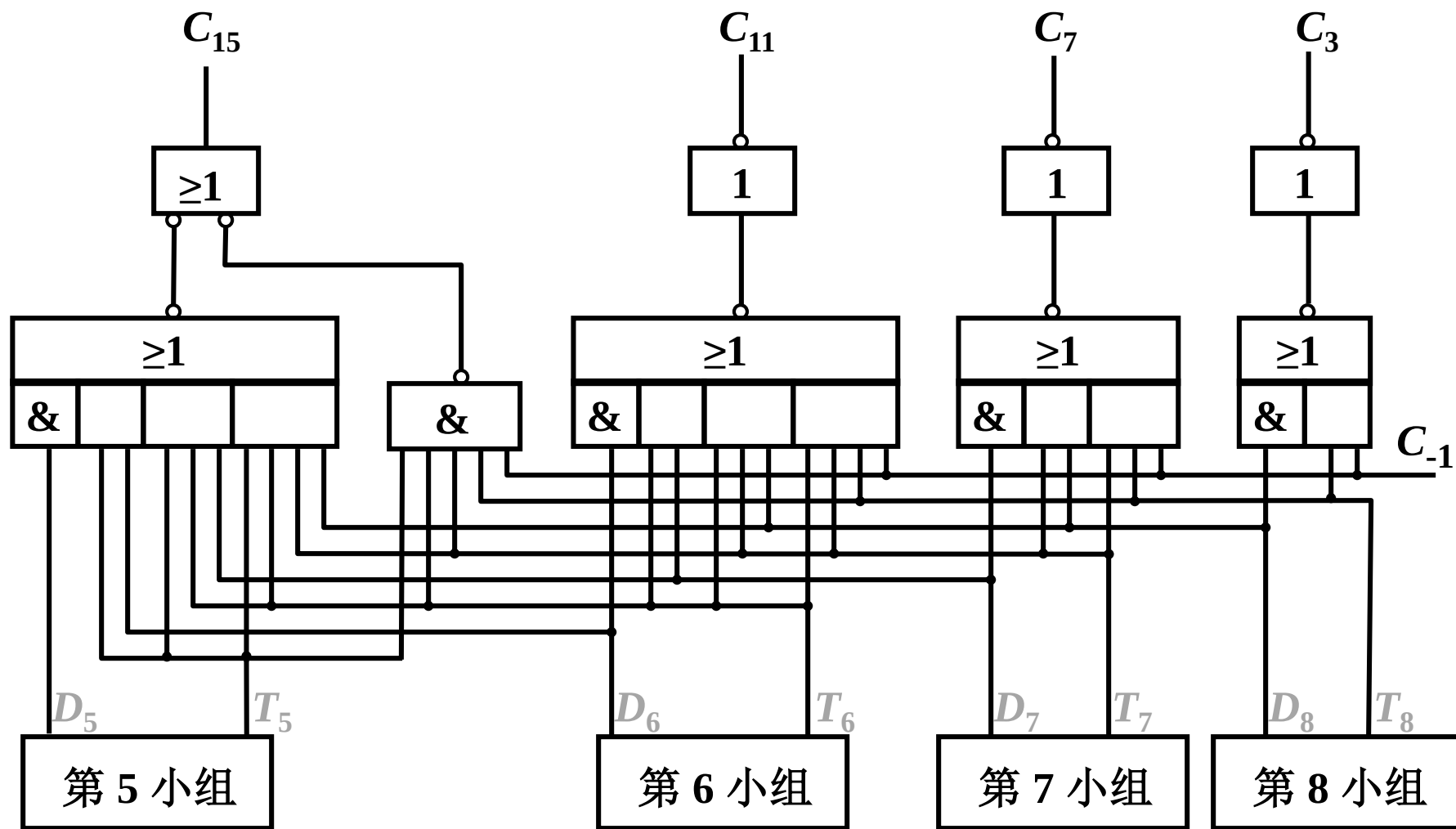
$$C_7 = D_7 + T_7 C_3 = D_7 + T_7 D_8 + T_7 T_8 C_{-1}$$

$$C_{11} = D_6 + T_6 C_7 = D_6 + T_6 D_7 + T_6 T_7 D_8 + T_6 T_7 T_8 C_{-1}$$

$$C_{15} = D_5 + T_5 C_{11} = D_5 + T_5 D_6 + T_5 T_6 D_7 + T_5 T_6 T_7 D_8 + T_5 T_6 T_7 T_8 C_{-1}$$

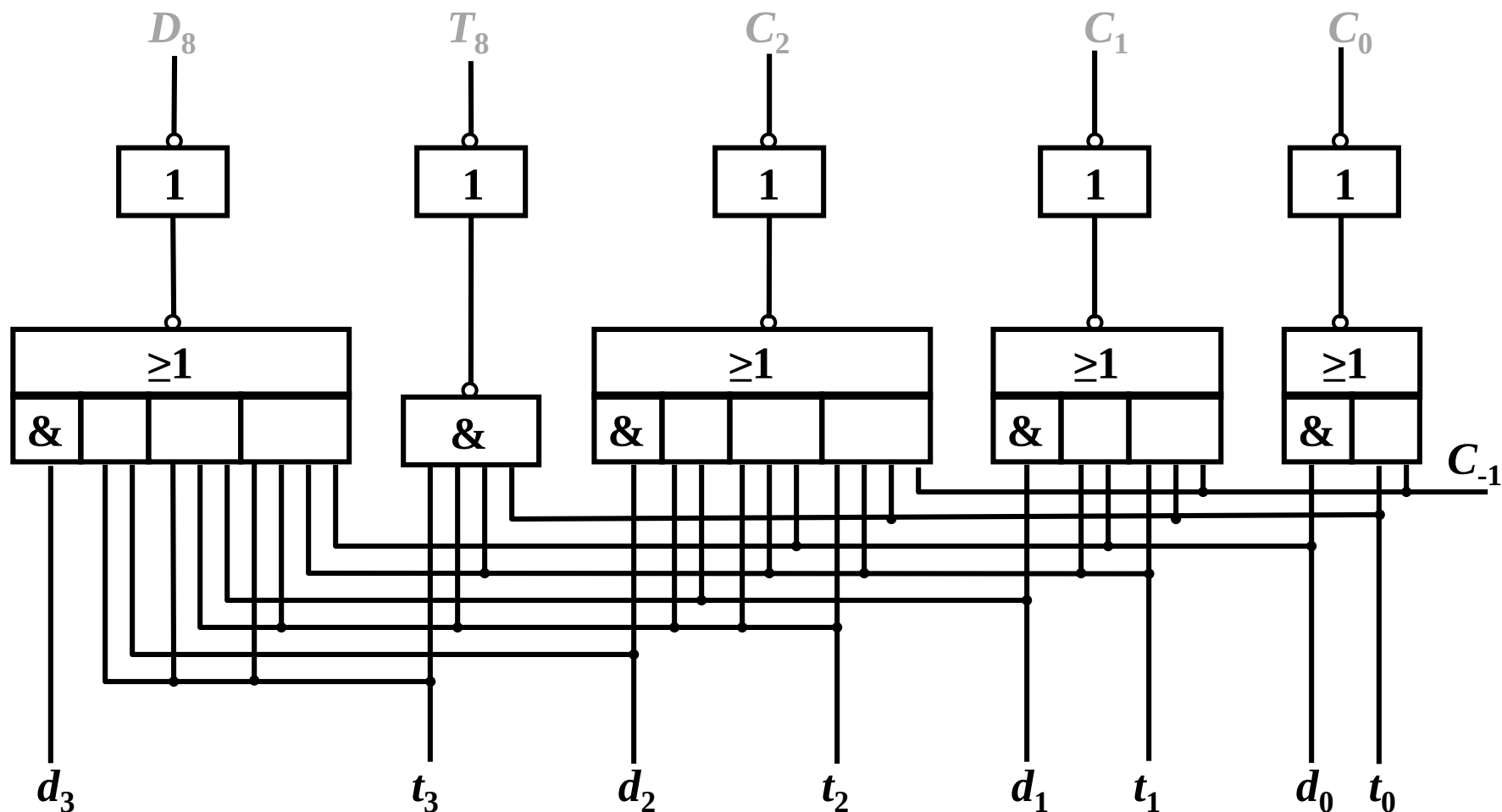
(4) 双重分组跳跃进位链的大组进位线路 6.5

以第 2 大组为例



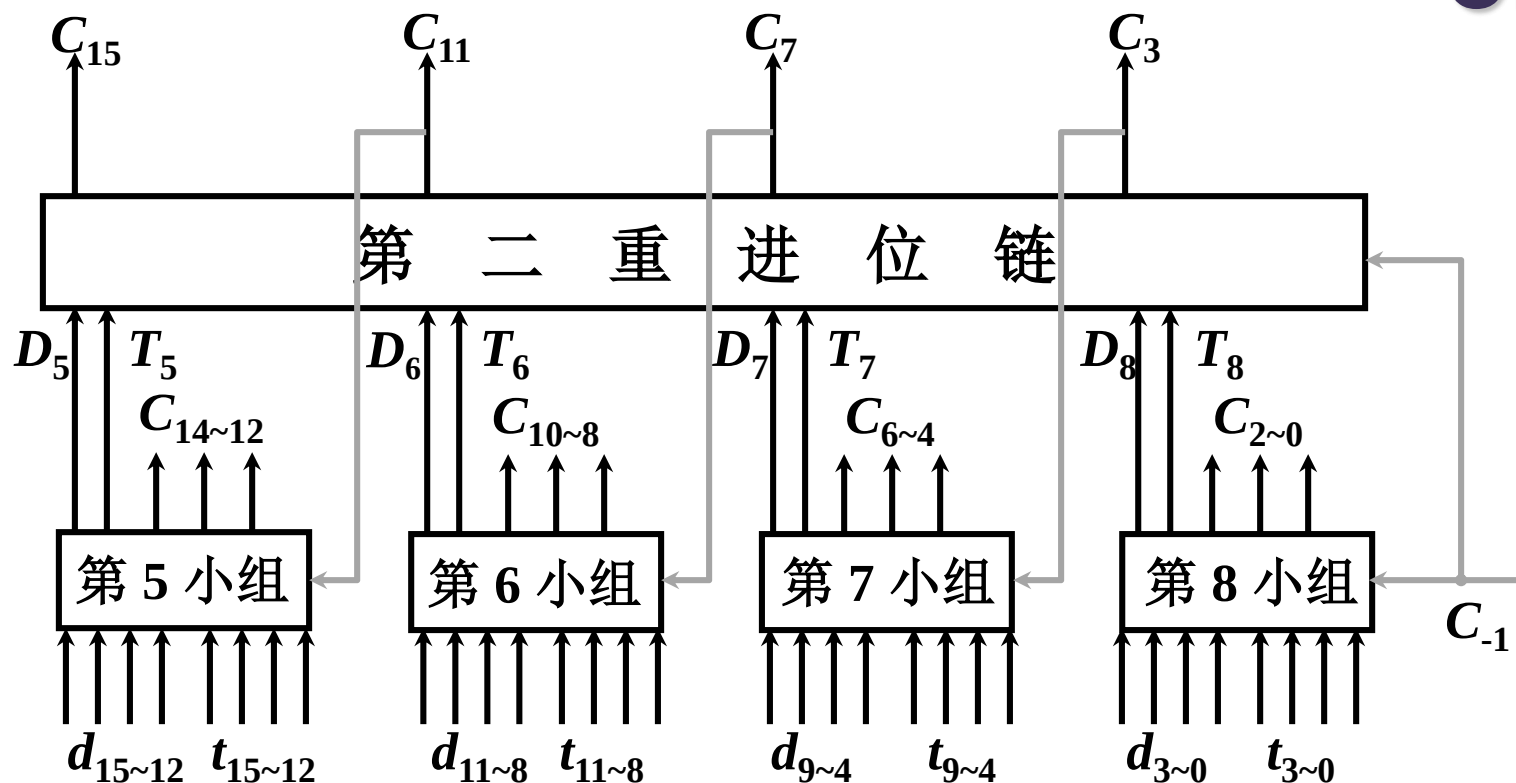
(5) 双重分组跳跃进位链的 小组 进位线路 6.5

以第 8 小组为例 只产生 低 3 位 的进位和 本小组的 $D_8 T_8$



(6) $n=16$ 双重分组跳跃进位链

6.5



当 d_i, t_i 和 C_{-1} 形成后

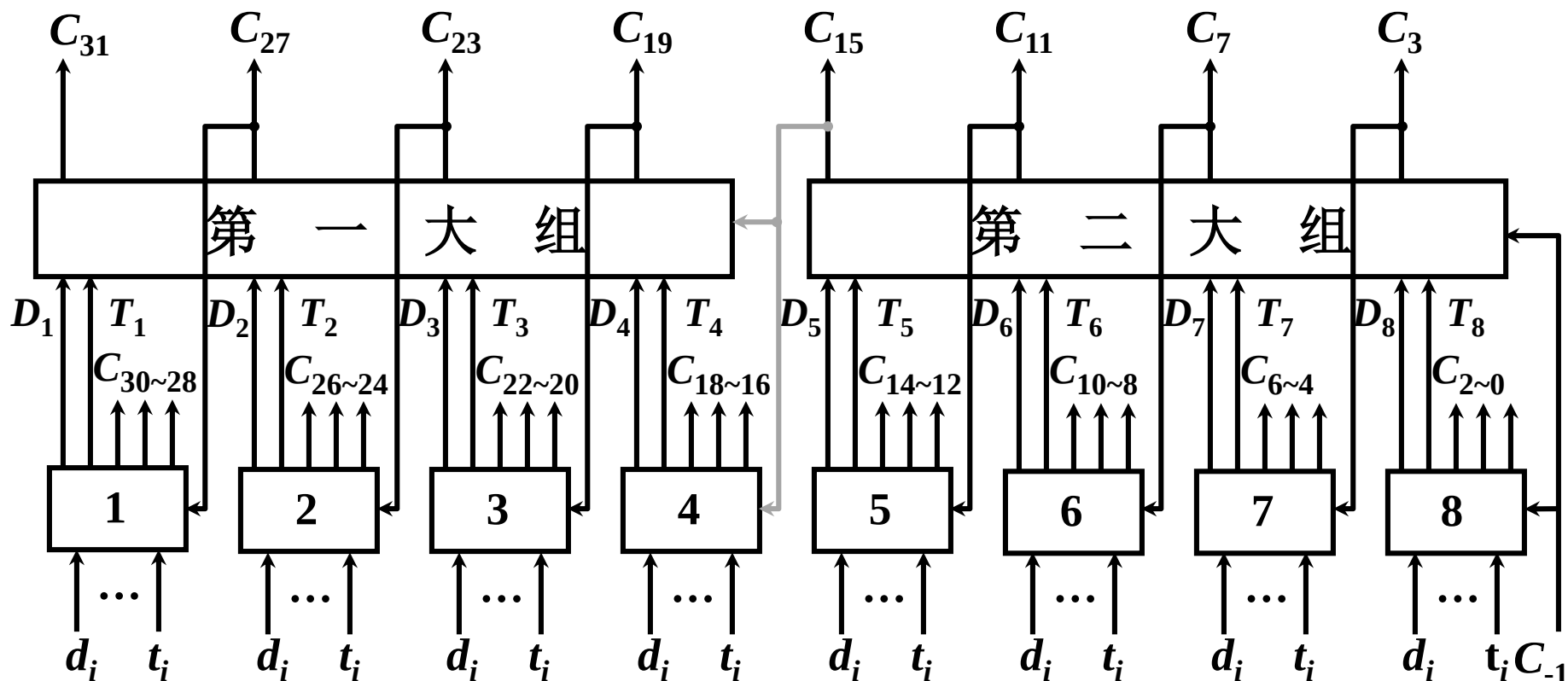
经 $2.5 t_y$	产生 $C_2, C_1, C_0, D_5 \sim D_8, T_5 \sim T_8$
经 $5 t_y$	产生 C_{15}, C_{11}, C_7, C_3
经 $7.5 t_y$	产生 $C_{14} \sim C_{12}, C_{10} \sim C_8, C_6 \sim C_4$

串行进位链 经 $32 t_y$ 产生 全部进位

单重分组跳跃进位链 经 $10 t_y$ 产生 全部进位

(7) $n=32$ 双重分组跳跃进位链

6.5



当 $d_i t_i$ 形成后 经 $2.5 t_y$ 产生 C_2 、 C_1 、 C_0 、 $D_1 \sim D_8$ 、 $T_1 \sim T_8$

$5 t_y$ 产生 C_{15} 、 C_{11} 、 C_7 、 C_3

$7.5 t_y$ 产生 $C_{18} \sim C_{16}$ 、 $C_{14} \sim C_{12}$ 、 $C_{10} \sim C_8$ 、 $C_6 \sim C_4$
 C_{31} 、 C_{27} 、 C_{23} 、 C_{19}

$10 t_y$ 产生 $C_{30} \sim C_{28}$ 、 $C_{26} \sim C_{24}$ 、 $C_{22} \sim C_{20}$